

Relatório Técnico do Sistema

Sistema IoT para Monitoramento e Controle de Ar-Condicionado

Este relatório descreve o sistema desenvolvido para monitoramento, simulação e controle inteligente de ambientes climatizados, com foco em salas de aula. A solução utiliza comunicação MQTT, containerização com Docker, automação com Node-RED, simulação de dispositivos ESP32 em C++ e uma interface web para acompanhamento e atuação manual.

O objetivo principal do sistema é representar uma arquitetura IoT distribuída capaz de coletar dados ambientais, processar regras de automação, controlar atuadores e disponibilizar os dados para visualização ou integração com plataformas externas, como a InterSCity.

1. Visão Geral

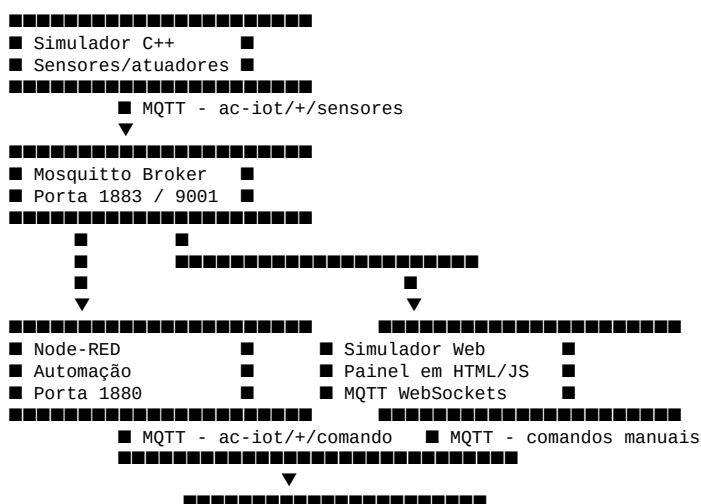
O sistema simula um conjunto de salas equipadas com sensores e atuadores. Cada sala possui dados como temperatura, umidade, luminosidade, presença humana, estado do ar-condicionado, estado da luz e modo de operação da automação.

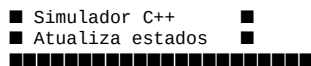
A comunicação entre os componentes é feita por meio do protocolo MQTT. O Mosquitto atua como broker central, recebendo mensagens de sensores e distribuindo comandos entre simulador, Node-RED e painel web.

Em termos práticos, o fluxo principal é:

1. O simulador C++ gera leituras de sensores para cada sala.
2. As leituras são publicadas no broker MQTT.
3. O Node-RED consome essas mensagens e aplica regras de automação.
4. O painel web acompanha os dados em tempo real e permite enviar comandos manuais.
5. O simulador recebe comandos MQTT, altera o estado interno da sala e publica novos dados.

2. Arquitetura do Sistema





A arquitetura foi organizada de forma modular, permitindo executar cada parte do sistema em containers ou por interface web local. O uso de MQTT desacopla os produtores de dados dos consumidores e facilita a expansão futura para dispositivos físicos ESP32.

3. Principais Componentes

3.1 Docker Compose

O arquivo `docker-compose.yml` é responsável pela orquestração dos serviços principais do sistema. Ele define containers, portas, dependências, volumes e variáveis de ambiente.

Serviços configurados:

Serviço	Função	Porta
mosquitto	Broker MQTT central	1883, 9001
nodered	Motor de automação e interface de fluxos	1880
simulador	Simulador C++ de salas e sensores	MQTT interno

O serviço `interscity` também possui estrutura pronta no projeto, mas no estado atual está comentado no `docker-compose.yml`. Ele pode ser ativado futuramente para encaminhamento de telemetria a uma API externa.

Variáveis importantes do simulador:

Variável	Descrição	Valor atual
MQTT_BROKER	Host do broker MQTT dentro da rede Docker	mosquitto
MQTT_PORT	Porta MQTT TCP	1883
PUBLISH_INTERVAL	Intervalo de publicação das leituras	1 segundo

3.2 Mosquitto MQTT Broker

O Mosquitto é o componente central de comunicação. Ele recebe publicações MQTT e entrega as mensagens aos clientes inscritos nos tópicos correspondentes.

Configuração principal:

- Porta 1883: comunicação MQTT TCP tradicional.
- Porta 9001: comunicação MQTT via WebSockets, usada pelo painel web.
- Persistência ativada em volume Docker.
- Logs gravados em volume próprio e também enviados para saída padrão.
- Conexões anônimas permitidas, o que simplifica a simulação local.

Arquivo de configuração:

`docker/mosquitto/config/mosquitto.conf`

Essa configuração permite que o simulador C++, o Node-RED e o painel web conversem pelo mesmo broker.

3.3 Simulador C++

O simulador está localizado em:

`simulador/main.cpp`

Ele representa dispositivos IoT em salas monitoradas. Atualmente, o sistema simula três salas:

- sala01
- sala02
- sala03

Cada sala possui os seguintes dados:

Campo	Descrição
id_sala	Identificador da sala
status_ac	Estado do ar-condicionado
setpoint_ac	Temperatura-alvo do ar-condicionado
status_luz	Estado da luz
temperatura	Temperatura ambiente
umidade	Umidade relativa
luminosidade	Nível de luz em lux
presenca	Indicação de presença humana
modo_ac	Modo da automação: ativo ou desativado
timestamp	Momento da geração da leitura

O simulador publica dados periodicamente em tópicos MQTT de sensores:

`ac-iot/sala01/sensores`
`ac-iot/sala02/sensores`
`ac-iot/sala03/sensores`

Exemplo de payload publicado:

```
{
  "id_sala": "sala01",
  "luminosidade": 31,
  "modo_ac": "ativo",
  "presenca": false,
  "setpoint_ac": 22.0,
  "status_ac": "desligado",
  "status_luz": "desligado",
  "temperatura": 25.42,
  "timestamp": 1779914042,
  "umidade": 40.47
}
```

O simulador também assina tópicos de comando:

`ac-iot/+comando`
`ac-iot/all/comando`

Ao receber comandos, ele atualiza seu estado interno e publica uma nova leitura da sala afetada. Isso permite que alterações feitas pelo Node-RED ou pelo painel web apareçam rapidamente no monitoramento.

Comandos aceitos:

Campo	Valores esperados	Efeito
comando	ligar, desligar	Liga/desliga o ar-condicionado

Campo	Valores esperados	Efeito
setpoint	Número	Atualiza o setpoint do ar-condicionado
luz	ligar, desligar	Liga/desliga a luz
temperatura	Número	Força temperatura simulada
umidade	Número	Força umidade simulada
luminosidade	Número inteiro	Força luminosidade simulada
presenca	true, false	Atualiza presença humana
modo_ac	ativo, desativado	Habilita/desabilita automação

3.4 Node-RED

O Node-RED é usado como motor de automação. Ele recebe as leituras publicadas pelo simulador, aplica regras de controle e publica comandos MQTT quando necessário.

Interface:

`http://localhost:1880`

Fluxos versionados:

`node-red/flows.json`

Fluxos usados pelo container:

`docker/nodered/data/flows.json`

O fluxo principal consome:

`ac-iot/+sensores`

E publica comandos em:

`ac-iot/<sala>/comando`

Regra de automação por presença

Uma das regras implementadas é o desligamento automático do ar-condicionado por ausência. A lógica atual funciona assim:

1. Quando há presença, o sistema registra o horário da última presença.
2. Quando a presença fica falsa, o Node-RED inicia a contagem.
3. Se a sala permanecer sem presença por 30 segundos e o AC estiver ligado, o Node-RED envia comando para desligar.
4. O log exibido no debug é:

AC desligado por ausência (30s)

Essa regra evita desligamentos imediatos e reduz ruído de logs, ao mesmo tempo em que mantém resposta mais rápida do que o tempo anterior de 1 minuto.

3.5 Simulador Web

O painel web está em:

simulador-web/index.html

Ele é uma interface HTML/JavaScript que se conecta ao Mosquitto usando MQTT via WebSockets na porta 9001.

Principais funções:

- Exibir salas descobertas dinamicamente por mensagens MQTT.
- Mostrar temperatura, umidade, luminosidade, presença, estado do AC e estado da luz.
- Enviar comandos manuais para uma sala específica.
- Enviar comandos globais para todas as salas.
- Forçar valores simulados de sensores.
- Marcar ou desmarcar presença humana.
- Ligar/desligar ar-condicionado e luz.
- Alterar setpoint do ar-condicionado.
- Exibir log de comandos MQTT enviados e recebidos.

O painel assina:

```
ac-iot/+/sensores
ac-iot/+/comando
ac-iot/all/comando
```

E publica comandos nos mesmos tópicos usados pelo Node-RED:

```
ac-iot/<sala>/comando
ac-iot/all/comando
```

Por usar WebSockets, o painel pode ser aberto diretamente no navegador, desde que o broker Mosquitto esteja em execução e expondo a porta 9001.

3.6 InterSCity

A pasta `intercity/` contém um serviço C++ preparado para atuar como ponte HTTP entre o ambiente local e a plataforma InterSCity.

Arquivos principais:

```
intercity/main.cpp
intercity/Dockerfile
intercity/config.example.env
```

Função prevista:

1. Receber telemetria via HTTP em `/telemetry`.
2. Validar e encaminhar os dados para uma API remota.
3. Usar token de autenticação quando configurado.
4. Disponibilizar rota `/health` para verificação de estado.

No estado atual do projeto, o serviço está implementado, mas o bloco correspondente está comentado no `docker-compose.yml`. Para uso efetivo, é necessário ativar o serviço e configurar o arquivo `.env` com `REMOTE_API_URL`, `REMOTE_API_PATH` e `API_TOKEN`.

3.7 Catálogo de Códigos IR

A pasta `ir-codes/` documenta a estrutura prevista para armazenar códigos infravermelhos de aparelhos de ar-condicionado.

Esses códigos são importantes para uma versão com hardware real, na qual um ESP32 com emissor infravermelho poderia enviar comandos físicos ao ar-condicionado.

Exemplos de comandos previstos:

- ligar
- desligar
- temperatura 20 °C
- temperatura 22 °C
- temperatura 24 °C

Essa etapa conecta a simulação atual ao uso real em campo.

4. Comunicação MQTT

O padrão de tópicos utilizado no sistema segue a estrutura:

`ac-iot/<id_sala>/<tipo>`

Onde:

- `<id_sala>` identifica a sala, por exemplo `sala01`.
- `<tipo>` indica se a mensagem é de sensores ou comando.

4.1 Tópicos de sensores

Tópico	Produtor	Consumidores
<code>ac-iot/sala01/sensores</code>	Simulador C++	Node-RED, painel web
<code>ac-iot/sala02/sensores</code>	Simulador C++	Node-RED, painel web
<code>ac-iot/sala03/sensores</code>	Simulador C++	Node-RED, painel web

4.2 Tópicos de comando

Tópico	Produtor	Consumidor
<code>ac-iot/sala01/comando</code>	Node-RED ou painel web	Simulador C++
<code>ac-iot/sala02/comando</code>	Node-RED ou painel web	Simulador C++
<code>ac-iot/sala03/comando</code>	Node-RED ou painel web	Simulador C++
<code>ac-iot/all/comando</code>	Node-RED ou painel web	Simulador C++

4.3 Exemplo de comando

```
{
  "id_sala": "sala01",
  "presenca": true
}
```

4.4 Exemplo de comando global

```
{  
  "comando": "desligar"  
}
```

Publicado em:

ac-iot/all/comando

5. Execução do Sistema

Para iniciar o ambiente:

```
docker compose up -d --build
```

Para verificar os serviços:

```
docker compose ps
```

Para acompanhar logs do simulador:

```
docker compose logs -f simulador
```

Para acessar o Node-RED:

<http://localhost:1880>

Para acessar o painel web:

[simulador-web/index.html](#)

Ou, no ambiente local usado durante o desenvolvimento:

<file:///Users/mangueira/mangueira-dev/ac-iot-sd-ufma/simulador-web/index.html>

6. Procedimento de Teste

Um teste funcional pode ser feito seguindo estes passos:

1. Subir os containers com Docker Compose.
2. Abrir o painel web.
3. Verificar se o painel indica conexão com o broker MQTT.
4. Aguardar a descoberta automática das salas.
5. Selecionar uma sala, por exemplo sala01.
6. Marcar presença humana.
7. Conferir se o campo Presença muda para Sim.
8. Ligar o ar-condicionado.
9. Desmarcar presença humana.
10. Aguardar 30 segundos.
11. Verificar no Node-RED ou no painel que o AC foi desligado por ausência.

Também é possível testar diretamente via terminal:

```
docker exec ac_iot_mosquitto mosquitto_pub \  
-h localhost \  
-t 'ac-iot/sala01/comando' \  
-m '{"presenca":true}'
```

E ler a próxima mensagem de sensores:

```
docker exec ac_iot_mosquitto mosquitto_sub \  
-h localhost \  
-t 'ac-iot/sala01/sensores' \  
-C 1
```

O retorno deve conter:

```
{  
  "presenca": true  
}
```

7. Persistência e Versionamento

O sistema possui dois pontos importantes de persistência:

1. Volumes do Mosquitto:

- mosquitto_data
- mosquitto_log

2. Volume do Node-RED:

- ./docker/nodered/data:/data

Os fluxos do Node-RED também são mantidos em:

node-red/flows.json

Isso permite versionar o fluxo no Git e, ao mesmo tempo, manter o ambiente Docker com os dados operacionais necessários.

8. Tecnologias Utilizadas

Tecnologia	Papel no sistema
Docker	Containerização dos serviços
Docker Compose	Orquestração local
Eclipse Mosquitto	Broker MQTT
MQTT	Protocolo de mensagens IoT
Node-RED	Automação e fluxo de controle
C++	Implementação do simulador e ponte InterSCity
Paho MQTT JavaScript	Cliente MQTT WebSocket no painel web
HTML/CSS/JavaScript	Interface web de simulação
nlohmann/json	Manipulação de JSON no C++
Boost.Beast	Cliente/servidor HTTP da ponte InterSCity

9. Estado Atual do Projeto

O sistema atual já permite:

- Executar infraestrutura MQTT local com Docker.
- Simular três salas.
- Publicar telemetria em tempo real.
- Controlar AC, luz, setpoint e presença.
- Visualizar dados pelo painel web.
- Executar automação de desligamento por ausência.
- Integrar fluxos com Node-RED.
- Preparar encaminhamento futuro para InterSCity.

Pontos ainda passíveis de evolução:

- Ativar o serviço InterSCity no Docker Compose.
- Implementar firmware real para ESP32.
- Substituir permissões MQTT anônimas por autenticação.
- Criar dashboards finais de indicadores energéticos.
- Adicionar testes automatizados de ponta a ponta.
- Atualizar documentação antiga para refletir os tópicos `ac-iot/...`

10. Conclusão

O sistema desenvolvido demonstra uma arquitetura IoT funcional, modular e extensível para monitoramento e controle de ar-condicionado em salas. A solução combina simulação, mensageria, automação e interface de operação em tempo real.

O uso do Docker facilita a reprodução do ambiente, enquanto o MQTT garante comunicação desacoplada entre componentes. O Node-RED permite criar regras de controle de forma visual e adaptável, e o painel web oferece uma forma simples de acompanhar e intervir no comportamento das salas simuladas.

Com a inclusão futura de dispositivos ESP32 reais e integração completa com a InterSCity, a arquitetura pode evoluir de uma simulação local para um sistema de monitoramento e controle aplicável em ambientes físicos.